MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

Anonymous Authors

***Abstract*—We study single-GPU mixture-of-experts inference under task memory caps that prevent full expert residency in HBM and host memory. SSD delivery then stalls computation. MECC combines profile-guided placement, cross-tier retention, and dependency-aware transfers. Its design preserves selected experts and FP16 weights. Illustrative targets specify 1.25–1.42× speedup over a local SSD-native adapter, rather than a reproduction of a published system. A single-GPU target takes eight seconds per token; a two-GPU resident target takes 2.8 seconds. The evaluation separates allocation feasibility from latency and nominal GPU-time. Measurements and matched external comparisons remain necessary to establish the design's benefit.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models select a subset of experts for each token [1]–[3]. Mixtral and DeepSeekMoE instantiate sparse routing in language models [4], [5]. A single GPU can execute selected experts without full HBM residency, provided each weight is delivered before computation.

Shared servers assign each service a host-memory allowance. When combined HBM and host residency cannot hold the expert pool, requests reach SSD. FlexGen and DeepSpeed Inference already plan heterogeneous-memory execution [6], [7]; flash-aware inference also addresses models exceeding DRAM [8]. We study expert delivery under fixed task caps.

Prior work uses CPU/GPU orchestration [9], memory-constrained planning [10], and SSD-native delivery [11]. MoE-Infinity traces activations for cache replacement and prefetch [12]. ProMoE predicts future expert use [13]. Learned replacement [14] and domain-guided residency [15] further exploit routes. We compare placement, retention, and scheduling under common physical caps.

Three coupled decisions govern expert delivery. **First, capacity has to be distributed across layers.** Uniform slots can leave some layers repeatedly reading from SSD. **Second, HBM eviction has to preserve later reuse.** A fixed CPU pool may discard useful experts, while independent tier caches can retain duplicate copies. **Third, residency has to track request changes.** An initial profile covers startup demand; online observations update that placement as the working set changes.

Our design treats the route profile as an initial residency prior and actual requests as online corrections. MECC assigns these roles across HBM, CPU pinned memory, and SSD. The intended compiler assigns expert rankings and layer capacities from an independent profile. The runtime promotes requested experts and demotes HBM victims into the host working set. Transfer overlap requires independent work and enforced readiness before computation. Figure 1 shows these interfaces; Section IV separates the design contract from the current prototype.

The technical question is whether profile-derived layer capacities and initialization improve finite-horizon service beyond the same exclusive retention and transfer machinery without that prior. Layer capacities remain fixed during a run; online updates change expert identities. This draft specifies that comparison and its resource accounting. Popularity placement, LRU, and event-based overlap are established mechanisms. Their composition alone does not establish novelty over coordinated-residency systems. All numerical outcomes below are authored scenarios.

II. Deployment Model and Challenges

A. Single-GPU Execution with Bounded Host Memory

Consider L MoE layers. Layer ℓ contains N_ℓ experts and selects k_ℓ per token. Expert $e = (\ell, j)$ is an immutable weight object of size s_e. The GPU also stores dense weights, activations, the KV cache, and temporary buffers. The expert-residency allowance is B_{H}; the total task-level host allowance is B_{C}. Host mirrors and staging buffers count toward B_{C}.

Let $\bar{B}_{\mathrm{H}}$ and $\bar{B}_{\mathrm{C}}$ denote unique expert-residency capacities after charging other allocations. The target regime satisfies $\sum_e s_e > \bar{B}_{\mathrm{H}} + \bar{B}_{\mathrm{C}}$ and exceeds the task host allowance. Post-initialization SSD demand traffic measures its operational impact; startup reads alone do not establish this regime. Models that fit in host memory serve as two-tier controls. Models that fit in HBM need no expert offloading.

An HBM hit uses resident weights. A CPU hit requires host-to-device (H2D) transfer. An SSD miss adds an SSD-to-host read. We record the expert's tier when demand arrives, before servicing the request. Separate counters record physical SSD and transfer traffic, including prefetches.

B. Challenge 1: Allocate Capacity across Layers

The value of a cache slot depends on the layer's request distribution and miss cost. A small working set needs few slots. A broader working set can cause repeated cold reads under the same allocation. This difference motivates moving capacity between layers while holding total memory fixed.

Figure 2(a) compares uniform and planned allocations on a held-out stream. Total slots, replacement, precision, and requests are fixed. The hypothetical plan assigns more slots to layers with higher cold-read pressure. Some donor layers incur more misses; the aggregate miss share falls.

C. Challenge 2: Preserve Reuse after HBM Eviction

An expert can be requested again soon after HBM eviction. A fixed CPU pool retains only its predefined members. Independent tier caches may instead retain copies of the same expert. These policies differ in both reuse coverage and physical duplication.

Figure 2(b) illustrates a fixed CPU pool, independent LRUs, and coordinated retention. It does not isolate reuse from duplicate storage. The missing control is exclusive LRU with matching per-layer combined capacities and initial stacks, accompanied by duplicate-byte and migration measurements. Figure 3 illustrates the state changes.

HYPOTHETICAL RESULTS DESIGN DRAFT — NOT MEASURED — NOT FOR SUBMISSION

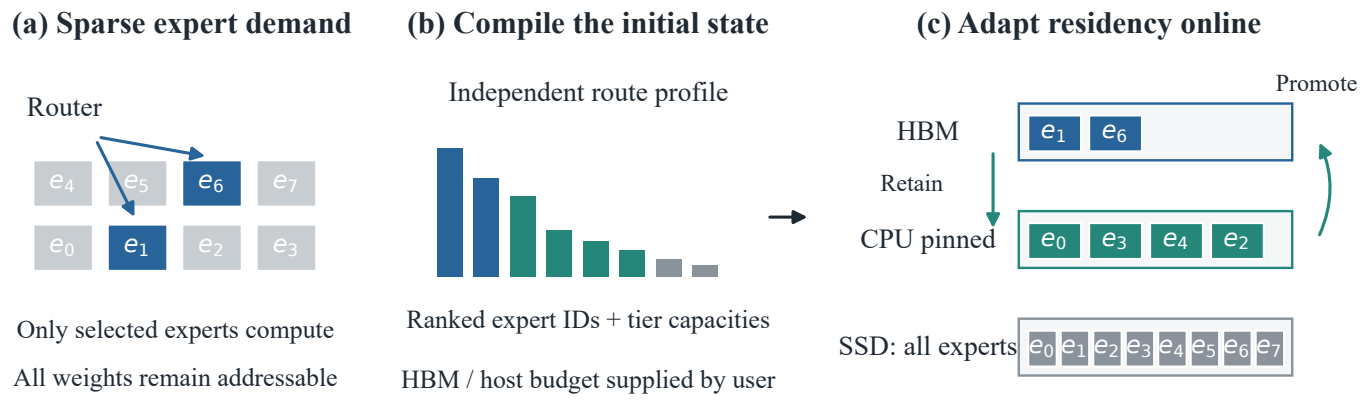


Fig. 1. MECC overview. (a) The router selects experts for computation. (b) An independent profile determines initial residency. (c) Promotion and demotion update the HBM and CPU working sets. SSD holds the backing weights. Expert IDs and weights remain unchanged. Schematic.

HYPOTHETICAL · NOT MEASURED

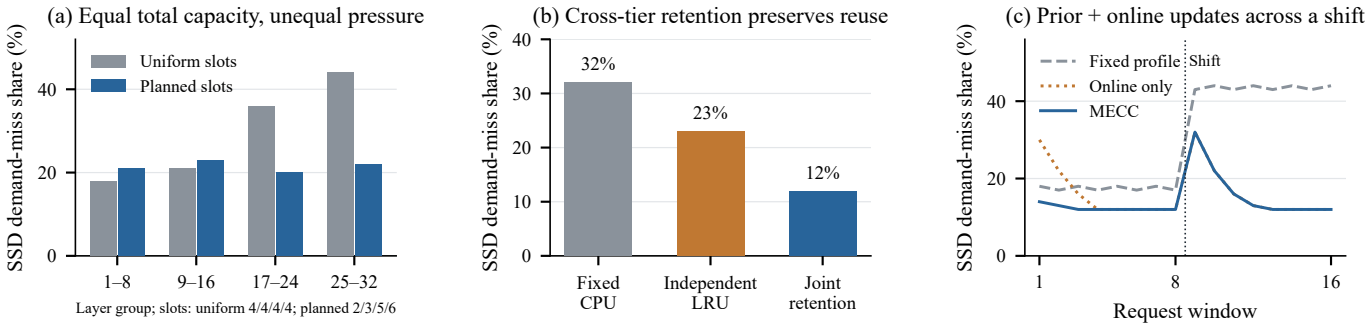


Fig. 2. Deployment challenges. (a) Layer allocation at fixed total capacity. (b) Retention rules under the same initial profile. (c) Startup and adaptation across a working-set shift. **All values and request windows are hypothetical.** The plots do not use measured MoE routes.

D. Challenge 3: Initialize Residency and Track Changes

A profile supplies access information before the first request. Online history tracks changes after service begins. Fixed placement can become stale when the working set shifts. An online policy without a profile first has to discover that working set. Their combination targets startup misses and later adaptation.

The required experiment crosses profile/random initialization with static/adaptive residency. Figure 2(c) illustrates three of these four configurations across an authored shift. The fourth configuration, profile cost, no-shift control, uniform access, and mismatched profiles remain unmeasured. These curves specify a hypothesis about startup and adaptation; they do not establish natural routing drift.

III. MECC Design

A. Compiler: Profile-Guided Placement

The compiler takes model metadata, memory budgets, delivery costs, and an independent route profile. The profile supplies request probability p_e for each expert. For equal-size experts within a layer, a discrete Zipf model summarizes the ranked probabilities:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{H_{N_\ell}(\alpha_\ell)}, \qquad H_n(\alpha) = \sum_{r=1}^{n} r^{-\alpha}. \tag{1}$$

Here $q_{\ell,r}$ is conditional on an expert selection in layer ℓ. Let w_ℓ be that layer's fraction of all selections. The global probability is $p_e = w_\ell q_{\ell,r}$, with $\sum_\ell w_\ell = 1$. For fixed top-k routing, $w_\ell = k_\ell / \sum_j k_j$. Each rank retains its profiled expert ID. Empirical probabilities can replace the fitted Zipf distribution. Neither representation encodes request order.

Let $x_{e,t} \in \{0,1\}$ indicate the primary tier of expert e. The tiers are HBM, CPU, and SSD, denoted by H, C, and S. Residency budgets $\tilde{B}_t$ must exclude buffers and mirrors. The following static objective is a reference model, not a claim that the current planner solves the full problem:

$$\begin{aligned} \min_x \quad & \sum_e p_e \sum_t x_{e,t} T_t(s_e), \\ \text{s.t.} \quad & \sum_t x_{e,t} = 1, \quad \sum_e s_e x_{e,t} \le \tilde{B}_t \quad (t \in \{\mathrm{H}, \mathrm{C}\}). \end{aligned} \tag{2}$$

The affine model $T_t(s) = a_t + s/b_t$ represents fixed overhead and effective bandwidth. Calibration uses the target machine. Equation 2 estimates static delivery cost under the profile. Runtime locality, migration, and overlap are evaluated separately.

Proposition 1: Assume equal-size objects, fixed request probabilities, independent tier capacities, and $T_\mathrm{H} \le T_\mathrm{C} \le T_\mathrm{S}$. An optimal static

placement fills HBM with the most probable objects, CPU with the next most probable objects, and SSD with the remainder.

Proof sketch. Suppose $p_i > p_j$ and object i occupies the slower tier. Exchanging the two objects changes expected cost by $(p_i - p_j)(T_{\text{fast}} - T_{\text{slow}}) \leq 0$. Repeated exchanges produce ranked placement. Filling an empty faster-tier slot also cannot increase cost. The result applies to the stated static model.

The multilayer implementation allocates slots by marginal coverage. Under the Zipf model, a slot covering rank r adds

$$\Delta_\ell(r) = w_\ell q_{\ell,r}. \tag{3}$$

This is a coverage score, not the exact latency marginal of Equation 2. Tier-boundary movements can affect both HBM and CPU assignments. The intended heuristic allocates HBM slots first, then CPU slots from the remaining ranks. At each step it selects the largest next-rank score; ties use layer ID. Expert-ranking ties use expert ID. It stops at the tier's slot cap or exhaustion of experts. Per-layer minima, if enabled, are assigned first and an infeasible minimum allocation must be rejected. A direct scan costs $O(L(S_{\text{H}} + S_{\text{C}}))$ after ranking, where S_t is a total slot cap. These validation rules describe the required planner contract; the prototype does not yet enforce all of them. Batched prefill additionally requires distinguishing token selections from distinct expert fetches. The sequential heuristic does not solve the general static objective.

The plan records per-layer capacities, ranked expert IDs, initial tiers, and modeled hit fractions. Model and profile identifiers track its inputs. If profiling is unavailable, the loader uses a labeled natural-order fallback. It initializes the runtime from the chosen plan.

B. Runtime: Cross-Tier Retention

Each layer has ordered HBM and CPU residency maps. An HBM hit updates recency. A CPU hit removes the expert from CPU residency and promotes it to HBM as most recently used. An SSD miss reads the weights into staging before H2D transfer. When HBM is full, its least-recently used expert is demoted to CPU. CPU pressure evicts the least-recently used host resident. The SSD backing weights remain available.

Figure 3 starts with HBM $[e_0, e_1]$ and CPU $[e_2, e_3, e_4]$. Requesting e_2 promotes it and demotes e_0. The next request for e_0 hits CPU. This example uses exclusive logical residency. An implementation can keep a CPU mirror to avoid device-to-host (D2H) transfer on eviction. The mirror consumes host memory and is charged separately.

For a serial demand stream within one layer, concatenate the HBM MRU-to-LRU stack with the CPU MRU-to-LRU stack. Exclusive promotion and victim demotion preserve the corresponding LRU stack of capacity $K_{\text{H}} + K_{\text{C}}$. This reference requires matching initialization, no prefetch or alternative admission, and completed transitions before the next request. It does not imply equivalence to a cache that reallocates capacity across layers, or dominance over static placement.

C. Delivery: Dependency-Aware Overlap

A correct asynchronous implementation requires a readiness event after each pinned-host copy. The computation stream must wait before consuming the weight. The source buffer must remain live until copy completion, and the resident weight until its last consumer finishes. These are required lifetime rules, including across separate loading and dispatch iterators; the prototype's current asynchronous path is not yet validated against them.

At batch one, routing reveals the selected IDs only after the current layer's gate. Copying one selected expert may overlap computation of another already-ready expert if kernel and aggregation order permit.

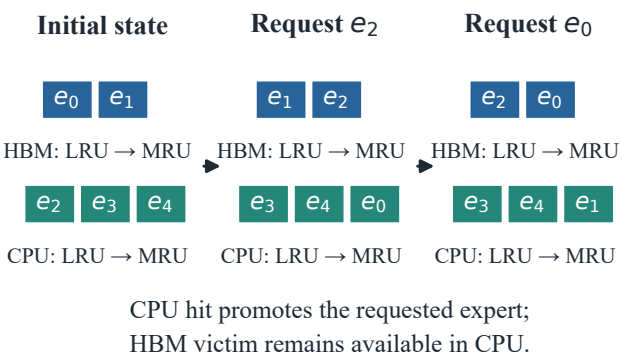


Fig. 3. Cross-tier retention with two HBM slots and three CPU slots. Lists run from LRU to MRU. A CPU hit promotes the requested expert and makes room for the HBM victim. Mirrors and staging consume additional physical space. Schematic.

This provides no knowledge of a later layer's route. In-flight weights must be protected from eviction. Replacement comparisons must disable predictive prefetch; the overlap control must hold admission, residency, and requests fixed. The available overlap and its exposed critical path remain measurements, rather than an assumed gain from a copy stream.

The design requires unchanged weights, selected experts, kernels, and aggregation order. Under these conditions, changing residency preserves the forward computation. The implementation also needs correct synchronization. The validation protocol compares synchronous and asynchronous paths on identical inputs using weight hashes, route selections, logits, tokens, and perplexity; these tests remain to be executed.

IV. Implementation

Model adapters map layer/expert IDs to weight objects, and policies share an end-to-end loader. The design requires separate state-machine and GPU synchronization checks. The current harness is not a validated implementation of every contract in Section III: synthetic layer exponents, optional co-occurrence ranking, CPU mirrors, asynchronous readiness, and phase-mixed counters require reconciliation before experiments. None of the GPU tests described here is reported as completed.

Physical accounting must deduplicate storage identities. GPU use includes dense weights, experts, KV/activations, and buffers. Host use includes residents, mirrors, staging, and other copies. A full CPU mirror implies host bytes at least equal to mirrored HBM expert bytes, before adding CPU residents. Thus the archived target of 42 GB of HBM experts with only 6 GB of mirrors is incompatible with full mirroring under a 40 GB host cap. We withdraw that breakdown as a feasible configuration. Residency must be reduced or a separately validated mirror-free path used; nominal caps alone do not certify any plotted point.

The measurement protocol requires separate timers and counters for initialization, warmup, prefill, decode, and quality evaluation. TPOT divides decode time by measured intervals; TTFT covers prefill and the first output token. Demand-miss share counts requests with no ready HBM or CPU copy at arrival, divided by all selected expert requests in that phase. In-flight prefetch is logged separately rather than counted as a ready hit. Physical SSD/H2D/D2H bytes use the same window and need not equal arrival-source counts. Current phase-mixed counters cannot supply this breakdown without changes.

HYPOTHETICAL RESULTS DESIGN DRAFT — NOT MEASURED — NOT FOR SUBMISSION

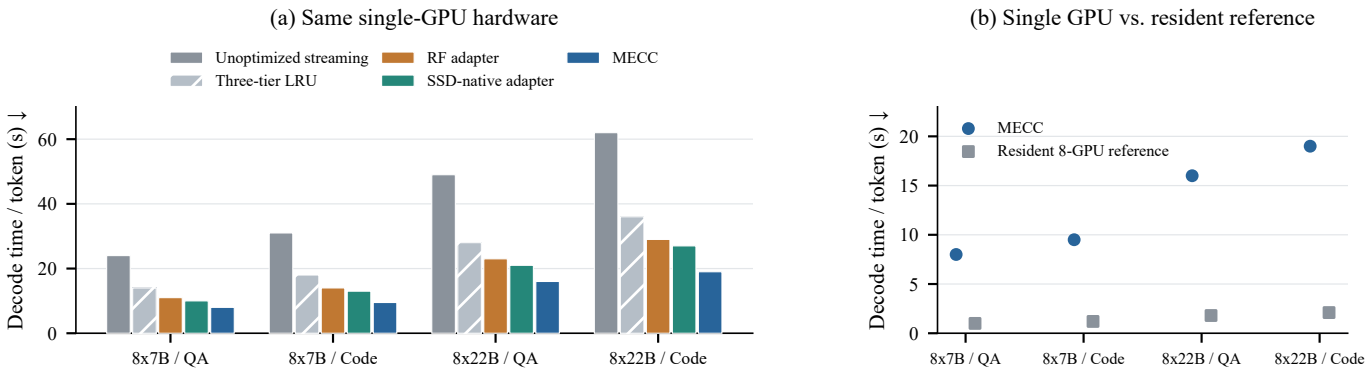


Fig. 4. **Hypothetical performance.** (a) Single-A100 policies with matched task budgets. RF and SSD-native denote local adapters, not published-system measurements. Targets have no error bars. Nominal allowances are 50 GB for experts and 40 GB for host memory; physical feasibility remains unverified. (b) Single-GPU MECC versus an eight-A100 resident reference. This comparison uses different GPU budgets.

V. Evaluation

Hypothetical evaluation. All results and quality checks below are authored targets. They are not measurements. Budget feasibility and implementation correctness are prerequisites, not established outcomes.

A. Experimental Methodology

Models and workloads. The scenario uses Mixtral-8x7B and Mixtral-8x22B [4], [16] with question-answering and code prompts. One A100 provides 80 GB of physical HBM. The nominal expert allowance is 50 GB, subject to total-peak checks. The task host cap is 40 GB. Profile and evaluation splits are independent. Main comparisons use batch one, 128 input tokens, 128 output tokens, and deterministic decoding. Dataset, model, and input revisions remain to be frozen for execution.

Unoptimized single-GPU baseline. The streaming baseline executes the same selected FP16 experts on one GPU. It shares the model, tokenizer, kernels, cold-read semantics, and task caps with MECC. It uses demand-driven staging. Profile placement, dynamic CPU retention, and transfer/computation overlap are all disabled. Default residency and achieved occupancy must be documented before this comparison is interpretable. This comparison measures the effect of the complete optimization stack.

Cache baselines. Three-tier LRU controls for basic caching. RF and SSD-native are hypothetical local mechanism adapters. The recency/frequency (RF) adapter uses a heuristic whose precise rule remains to be frozen; FlashMoE [14] learns a predictor from these signals. SSD-native represents three-tier residency motivated by SSD-LLaMA [11]. Neither establishes a reproduction of the cited system. Versioned external runs, including MoE-CORE [15], remain separate experiments.

Multi-GPU reference. The resident reference partitions the full FP16 model across two, four, or eight A100s where capacity permits. It uses matched inputs and decoding without SSD delivery. Its resource record includes GPU count, parallelism, interconnect, software revision, and memory peak. The eight-GPU point is an authored scenario reference, independent of the cited systems.

Metrics. We report TPOT, TTFT, throughput, demand-miss share, SSD/H2D/D2H bytes, and physical memory peaks. GPU-time comparisons use the same workload. We omit authored repeat variation from the plots. Execution must report independent repeats, request-level distributions, and the statistic used for uncertainty. No uncertainty estimate is available in this draft.

B. Latency under Matched Single-GPU Budgets

Figure 4(a) fixes one A100 and the task host cap across four model/workload combinations. Scenario TPOT is 8.0, 9.5, 16.0, and 19.0 s for MECC, versus 24, 31, 49, and 62 s for streaming. These values correspond to 3.00×, 3.26×, 3.06×, and 3.26× speedup. The comparison measures the full placement, retention, and delivery stack. Section V-D illustrates cumulative additions; independent attribution remains untested.

The SSD-native adapter takes 10, 13, 21, and 27 s per token. Relative to this cache baseline, MECC gains 1.25–1.42×. The three-tier-LRU comparison further separates basic caching from profile placement and coordinated delivery.

Scenario TTFT is 60/68/96/112 s for MECC and 130/155/230/280 s for streaming. These values describe first response. TPOT describes later token intervals. Initialization is charged separately in the cost analysis below.

C. Latency versus GPU Allocation

Figure 4(b) gives eight-GPU TPOT of 1.0/1.2/1.8/2.1 s. The resident reference reduces latency at a higher GPU allocation. The single-GPU point serves applications whose latency allowance permits this tradeoff.

Table I includes every specified resident allocation. The two-GPU resident target is faster and uses fewer nominal GPU-seconds than single-GPU MECC. The one-GPU configuration instead tests feasibility when only one GPU can be allocated. A suitable use case is offline generation with a declared latency allowance; 127 decode intervals at 8 s take about 17 minutes. This does not establish GPU-time, energy, or monetary savings.

D. Component and Mechanism Analysis

Figure 5 adds components to streaming on the first workload. Profile placement reduces scenario TPOT from 24 to 18 s. Retention reduces it to 11 s. Overlap reduces it to 8 s. Each change measures a component within that stack. Component interactions require additional leave-one-out and factorial comparisons.

SSD demand-miss shares are 46%, 33%, 20%, and 20%. Placement and retention reduce the illustrated SSD demand share. Overlap

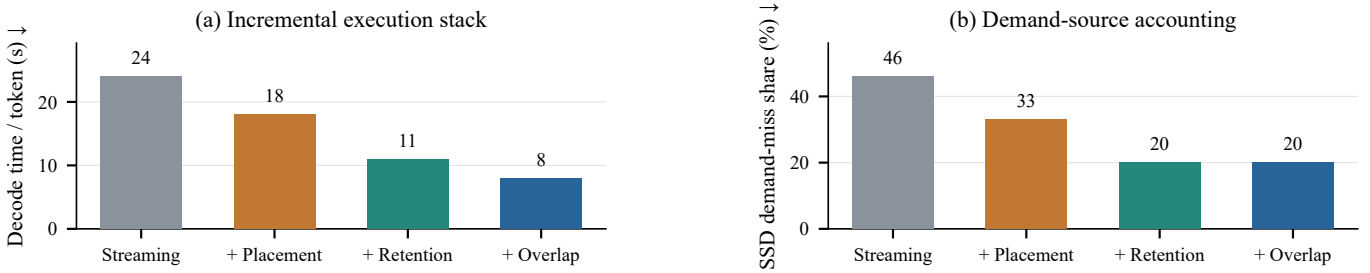


Fig. 5. **Hypothetical ablation.** (a) TPOT after adding each component. (b) SSD demand-miss share. The overlap-only step fixes admission, residency, and requests. Its miss share stays at 20%; exposed transfer time decreases.

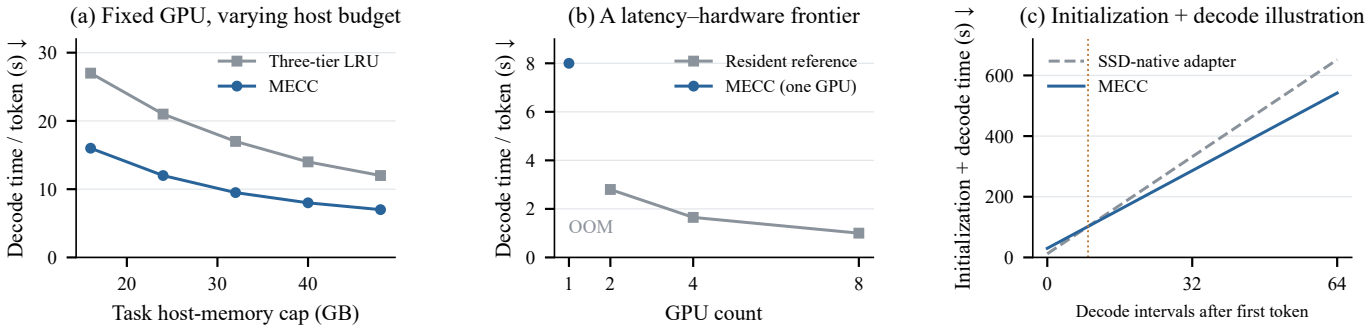


Fig. 6. **Hypothetical sensitivity.** (a) TPOT versus total task host cap. (b) Allocation comparison on 8x7B question answering; the full-model-resident reference exceeds one GPU. Only the single-GPU MECC design is specified. (c) Initialization plus decode intervals. An 18 s initial overhead and a 2 s/interval saving yield equality at nine intervals. Additional loading, profiling, and prefill costs are outside this illustration.

TABLE I
HYPOTHETICAL 8X7B DEPLOYMENT POINT; NOT MEASURED.

Configuration	TPOT (s)	GPU-s/token
MECC, 1 GPU	8.0	8.0
Resident, 2 GPUs	2.8	5.6
Resident, 4 GPUs	1.65	6.6
Resident, 8 GPUs	1.0	8.0

keeps the same miss share and reduces exposed waiting. The 12% target in Figure 2(b) belongs to a separate controlled trace.

Mechanism controls hold requests fixed. Separate controls must vary layer allocation and initial expert identities while holding the other fixed, and charge initial transfers. The retention comparison must record duplicate bytes under each policy. Within-phase shuffling preserves expert histograms and tests sensitivity to order. The shift scenario characterizes an authored workload; it provides no measured evidence of natural MoE drift.

E. Resource Sensitivity, Cost, and Quality

Figure 6(a) increases the host cap from 16 to 48 GB on a fixed GPU. Scenario TPOT falls from 16 to 7 s for MECC and from 27 to 12 s for conventional LRU. The paired curves compare residency at each task allowance. Each point requires independent physical accounting, including mirrors and buffers. These target curves do not establish feasibility under the stated cap.

Figure 6(b) varies GPU count for 8x7B question answering. The resident configuration does not fit on one GPU. Its two-, four-, and eight-GPU targets are 2.8, 1.65, and 1.0 s/token. Only the single-GPU MECC target is shown. Earlier multi-GPU MECC targets are withdrawn from the figure because this draft does not specify that extension. The smallest feasible resident reference must remain in the comparison.

Figure 6(c) adds initialization to decode time. Initialization costs 30 s for MECC and 12 s for the SSD-native adapter. Decode costs 8 and 10 s/token, respectively. The plotted terms are equal at nine decode intervals after the first token. This is not a full-request crossover. For n output tokens, full cost is $J(n) = I + \mathrm{TTFT} + (n-1)\mathrm{TPOT}$, with amortized profiling/loading included in I and no double counting. The adapter TTFT and profile costs are unspecified, so no numerical full-request break-even follows.

The complete sensitivity matrix also varies HBM capacity, input/output lengths, batch size, and profile coverage or mismatch. Leave-one-out controls separate zero-copy eviction, pinning, batch H2D, and prefetching. A reverse resource comparison fixes a TPOT limit and finds each policy's minimum feasible host cap. These cases remain planned; the current curves cover only the displayed scenario points.

The quality targets are identical weight hashes and expert selec-

tions, zero logit difference from deterministic synchronous execution, and zero perplexity difference on a fixed corpus. These checks remain hypothetical. Nondeterministic kernels require prespecified error tolerances and token comparisons. Weight checks and execution checks test separate properties.

VI. Related Work

Sparse execution and resident serving. Switch and GShard separate active computation from total parameters [2], [3]; DeepSeekMoE uses fine-grained and shared experts [5]. DeepSpeed-MoE optimizes distributed MoE inference [17], while Tutel adapts parallelism and pipelining [18]. MegaBlocks targets sparse training kernels [19], not SSD expert delivery. PagedAttention manages KV allocation [20]; SGLang reuses prefix KV state [21]. These address distinct bottlenecks and do not replace expert-residency comparisons.

Heterogeneous memory. ZeRO-Infinity uses GPU, CPU, and NVMe for training [22]. DeepSpeed Inference extends heterogeneous execution to serving [7]. FlexGen plans tensor placement and schedules for throughput-oriented inference [6]. LLM in a flash combines reuse windows with flash-friendly weight layout [8]. PowerInfer places hot neurons on GPU and computes cold neurons on CPU [23]. These precedents motivate separating byte traffic, execution location, and batching effects.

Offloading. Fast MoE offloading [24] and SwapMoE [25] study memory-constrained serving. Fiddler [9] initializes GPU residency from expert popularity and executes other experts on CPU. MoE-Lightning [10] searches batching, execution, and weight/KV placement under CPU/GPU memory constraints. Its search excludes disk variables; MECC targets SSD demand.

Expert prediction. Pre-gated MoE introduces a pre-gating function through algorithm–system co-design [26]. ProMoE uses intermediate results to predict later expert use [13]. MoE-Infinity traces sparse activations to guide replacement and prefetch [12]. The demand-only retention control excludes prediction; the overlap control can use only IDs available from the unchanged router. Prediction accuracy, timing, and additional storage require separate accounting.

SSD-based serving. SSD-LLaMA [11] combines expert-pack I/O, lossless transport, and RAM/VRAM caches. Frequency and recent use guide VRAM admission; transient experts can bypass retention. RAM uses LRU. FlashMoE [14] trains a feed-forward distance predictor from normalized recency/frequency signals and Belady-derived labels. Ordinary RF or LRU omits these mechanisms.

Coordinated residency. MoE-CORE [15] uses hit-rate curves for layer allocation, domain profiles for initialization, and routing history/scores for replacement. Optional expert substitution is separate from its exact path. ZipMoE [27] combines lossless bit-field compression, compression-aware caching, and cache-affinity scheduling. MECC evaluates uncompressed initialization and cross-tier retention under matched expert semantics and physical caps.

The scope is single-GPU serving with SSD demand under task-level memory caps. The hypothetical local-adapter comparisons do not establish superiority over external systems. Submission requires replacing all scenario outcomes with measurements.

VII. Conclusion

MECC addresses SSD weight delivery during single-GPU MoE inference. A compiled profile assigns initial residency. Online retention updates the working set, and dependency-aware transfers deliver selected FP16 experts. The hypothetical evaluation separates latency gains at fixed single-GPU budgets from the latency paid to reduce GPU allocation. The resource, mechanism, and quality comparisons define the remaining measurements.

Content Generated by AI

OpenAI Codex (GPT-6) assisted with text in all sections, figure-generation code, and hypothetical outcomes. Human authors are responsible for verifying the content, implementation, references, and replacement measurements before submission.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[4] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[5] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[6] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[7] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[8] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[9] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[10] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[11] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[12] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[13] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[14] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[15] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[16] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[17] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[18] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[19] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[20] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[21] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[22] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[23] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[24] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[25] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[26] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[27] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

HYPOTHETICAL RESULTS DESIGN DRAFT — NOT MEASURED — NOT FOR SUBMISSION